

A Learned Index for Exact Similarity Search in Metric Spaces

ABSTRACT

Indexing is an effective way to enable efficient data access and query processing in large databases. Traditional indexing techniques make no assumptions about data distribution, which may cause large time and storage costs. Recent efforts have adopted the concept of *learned index* to replace traditional index structures with machine learning models to address these problems. However, accurate and efficient processing of similarity queries in high-dimensional metric spaces remains to be a challenge. In this paper, we propose a novel index structure called LIMS that integrates compact-partitioning methods, pivot-based techniques and learned indexes to support efficient similarity query processing in metric spaces. LIMS partitions the underlying data into clusters where each of them follows a relatively uniform data distribution. Data redistribution is achieved by utilizing a few effective pivots for each cluster. Similar data are mapped into compact regions and the mapped values are totally ordinal. Machine learning models are developed to approximate the position of each data record on the disk. Efficient LIMS-based algorithms are designed for processing range queries and nearest neighbor queries, as well as for dynamic updates. Extensive experiments with real-world and synthetic data demonstrate the superiority of LIMS compared with traditional indexes and state-of-the-art learned indexes.

1 INTRODUCTION

Similarity search is one of the fundamental operations in the era of big data. It finds the objects from a large database within a distance threshold to a given query object (called *range queries*) or the top- k most similar to the query object (called *k nearest neighbor queries*, or *kNN queries*), based on certain similarity measures or distance functions. For example, in spatial databases, a similarity query can be used to find all restaurants within a given range in terms of *Euclidean distance*. In image databases, similarity queries can be used to find the top 10 most similar images to a given image in terms of *Earth mover's distance* [28]. To accommodate a wide range of data types and distance functions, we consider similarity searches in the context of metric spaces in this paper. A metric space is a general space which makes no requirement of any particular data representation, but only a distance function that satisfies the four properties, namely non-negativity, identity, symmetry and triangle inequality (Definition 1, Section 3). A number of metric-space indexing methods have been proposed in the literature to accelerate similarity query processing [4, 6, 13, 27, 37]. These indexing methods that are based on tree-like structures are increasingly challenged by the volume and complexity of data. On one hand, query processing with such indexes requires traversing many index nodes (*i.e.*, nodes on the path from the root node to a leaf node) in the tree structure, which can be time-consuming. On the other hand, tree-like indexes impose non-negligible storage pressure on datasets that store complex and large objects, such as image and audio data.

In recent years, the concept of *learned index* [18] has been developed to provide a new perspective on indexing. By enhancing or even replacing traditional index structures with machine learning models that can reflect the intrinsic patterns of data, a learned index can look up a key quickly and at the same time save a lot of memory space required by traditional index structures. The original idea is limited to single dimensional case where data is sorted in an in-memory dense array. Directly adapting this idea for multi-dimensional case is unattractive, since multi-dimensional data has no natural sort order. Several multi-dimensional learned index structures have been proposed to address these issues [9, 10, 17, 19, 21, 26, 34, 36] (detailed discussions refer to Section 2). Despite the significant success of these learned indexes compared with traditional indexing methods, they still have some limitations. First of all, existing learned index structures do not support similarity searches in a generic metric space. They are designed for vector spaces, where some nice properties such as coordinate structure and dimension information can be used. Such special properties are not available in a generic metric space. The generality of metric space provides an opportunity to develop unified indexing methods. The lack of coordinate and dimension information, however, presents a big challenge to develop an efficient learned indexing method. Secondly, existing learned multi-dimensional index structures suffer from the phenomenon called *curse of dimensionality* when the number of dimensions grow. Many indexing methods that extend the traditional multi-dimensional indexes are restricted to certain types of data space partitioning, which inevitably leads to rapid performance degradation as the number of dimensions increases. Third, some existing learned indexes cannot return accurate query results because of errors caused by machine learning models.

To address the aforementioned limitations, we develop a novel disk-based learned index structure for metric spaces, called LIMS, to facilitate exact similarity queries (*i.e.*, point, range and *kNN* queries). In contrast to coordinate-based data partitioning, LIMS adopts a distance-based clustering strategy to group the underlying data into a number of subsets, in order to decompose complex and potentially correlated data into clusters with simple and relatively uniform distributions. LIMS selects a small set of pivots for each cluster and utilizes the distances to the pivots to perform data redistribution. This reduces the dimensionality of the data to the number of pivots adopted. By using a proper pivot-based mapping, LIMS organizes similar objects into compact regions and imposes a total order over the data. Such organization can significantly reduce the number of distance computations and page accesses during query processing. In order to further boost the search performance, LIMS follows the concept of *learned index*, using several simple polynomial regression models to enable quick locating of data records that might match the query filtering conditions. As we will show later, LIMS significantly outperforms other multi-dimensional learned indexes and traditional indexes in terms of average query time and the number of page accesses when processing high dimensional data.

In summary, this paper makes the following contributions.

1. We design LIMS, the first learned index structure for metric spaces, to facilitate exact similarity search.
2. Efficient algorithms for processing point, range and k NN queries are proposed, enabling a unified solution for searching complex data in a representation-agnostic way. An update strategy is also proposed for LIMS.
3. To the best of our knowledge, no experiment evaluation between different types of learned indexes has been performed. In this paper, we compare four multi-dimensional learned indexes. Extensive experiments with real-world and synthetic data demonstrate the superiority of LIMS.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 introduces basic concepts and formulates the research problem. Section 4 describes the details of LIMS. LIMS-based similarity query algorithms are discussed in Section 5. Section 6 reports the experimental results. Section 7 concludes the paper.

2 RELATED WORK

Due to space limitations, we focus on reviewing learned multi-dimensional indexes here. Good surveys of various traditional metric-space indexing methods can be found in [4, 6, 13, 27, 37].

The idea of *learned index* is that indexes can be regarded as models which take a key as the input and output the position of the corresponding record. If such a ‘black-box’ model can be learned from data, a query can be processed by a function invocation in $O(1)$ time instead of traversing a tree structure in $O(\log n)$ time. RMI is the first to explore how to enhance or replace classic index structures with machine learning models [18]. It assumes that data is sorted and kept in an in-memory dense array. In light of this, machine learning model essentially is to learn a cumulative distribution function (CDF). RMI consists of a hierarchy of models, where internal nodes in the hierarchy are the models responsible for predicting the child model to use, and a leaf model predicts the position of record. Since RMI utilizes the distribution of data and requires no comparison in each node, it provides significant benefits in terms of storage consumption and query processing time. For the sake of quickly correcting errors caused by machine learning models and supporting range query, RMI is limited to index key-sorted dataset, which makes a direct application of RMI to multi-dimensional data infeasible because there is no obvious ordering of points. Even if these points are embedded into an ordered space, guaranteeing the correctness and efficiency of query (e.g., range query and k NN query) remains a challenging task.

ZM index is the first effort to apply the idea of *learned index* to multiple dimensional spaces [34]. It adopts the Z-order space filling curve [22] to establish the ordering relationship for all points and then invokes RMI to support point and range queries. The correctness is guaranteed by a nice geometric property of z-order curve, i.e., monotonic ordering. However, ZM index may need to check many irrelevant points during refinement phase, which would get worse for high dimensional spaces. It does not support k NN queries and index updates. ZM will be one of our baselines.

Recursive spatial model index (RSMI) builds on RMI and ZM [26]. It develops a recursive partitioning strategy to partition the original space, and then groups data according to predictions. This results in a learned point grouping, which is different from RMI that fixes

the data layout first and trains a model to estimate positions. For each partition, RSMI first maps points into rank space and then invokes ZM to support point, range and k NN queries. However, the correctness of range and k NN queries can not be guaranteed. In addition, since RSMI is still based on space filling curves, the good performance of RSMI is confined to low dimension spaces.

LISA [19], a learned index structure for spatial data can effectively reduce the number of false hits compared with ZM, by 1) partitioning the original space into grid cells based on data distribution; 2) ordering data with a partially monotonic mapping function and rearranging data layout according to mapped values; 3) decomposing a large query range into multiple small ones. LISA has a dynamic data layout as RSMI, but the correctness of range and k NN queries can be guaranteed by the monotonicity of the models. However, its advantage in low scan overhead comes with the costly checking procedure and high index construction time. And the grid-based partitioning strategy makes it unsuitable for high dimensional spaces. Besides, LISA-based k NN query processing suffers from many repeated page accesses due to doing range queries with increasing radius from the scratch. LISA will be also one of our baselines.

Similar to LISA, Flood [21] also partitions data space into grid cells along dimensions such that for each dimension, the number of points in each partition is approximately the same. Flood assumes a known query workload and utilizes sample queries to learn an optimal combination of indexing dimensions and the number of partitions. Once these are learned, Flood maintains a table to record the position of the first point in each cell. At query time, Flood invokes RMI for each dimension to identify the cells intersecting the query and look up the cell table to locate corresponding records. However, Flood’s grid cannot efficiently adapt to correlated data distribution and skewed query workloads. Tsunami [10] extends Flood by utilizing query skew to partition data space into some regions, and then further dividing each region based on data correlations. However, simply choosing a subset of dimensions could degrade the performance as dimensionality increases. These studies are not discussed further as we do not assume known query workload.

A multidimensional learned (ML) index [9] combines the idea of iDistance [15] and RMI. It first partitions data into clusters, and then identifies the cluster center as the reference point. After all data points are represented in a one-dimensional space based on the distance to the reference point, RMI can be applied. Different from iDistance, ML uses a scaling value rather than a constant to stretch the data range. However, points along a fixed radius also have the same value after the transformation, leading to many irrelevant points to be checked. Besides, it does not support data updates. ML will also be one of our baselines.

Different from finding a sort order over multi-dimensional data and then learning the CDF, a reinforcement learning based R-Tree for spatial data (RLR-Tree) [12] uses machine learning techniques to improve on the classic R-Tree index. Instead of relying on hand-crafted heuristic rules, RLR-tree models two basic operations in R-tree, i.e., choosing a subtree for insertion and splitting a node, as *Markov decision process* [25], so reinforcement learning models can be applied. Because it does not need to modify the basic structure of the R-Tree and query processing algorithms, it is easier to be deployed in the current databases systems than the concept of

Table 1: List of key notations

Notation	Description
P	The dataset
$\mathbb{U}, dist$	The data space, distance metric
$\mathbb{R}^*, \mathbb{N}^*, d$	Positive real number set, positive integer set, dimensionality
p, q	A data point, a query point
r	Query radius
k	The number of nearest neighbors
K	The number of clusters
m	The number of pivots
N	The number of super rings
C_i	The i th cluster
$O_j^{(i)}$	The j th pivots in i th cluster
$dist_max_j^{(i)}$	The distance of the furthest object in i th cluster from the j th pivot
$dist_min_j^{(i)}$	The distance of the nearest object in i th cluster from the j th pivot

learned index. However, due to the curse of dimensionality, the minimum bounding rectangle for a leaf node (even in an optimal R-tree) can be nearly as large as the entire data space, such that the R-tree becomes ineffective. Similar to RLR-Tree, Qd-Tree [36] uses reinforcement learning to optimize the data partitioning strategy of kd-tree based on a given query workload, and suffers from the same problem. These studies are not discussed further since they are out of our scope.

3 BACKGROUND

In this section, we first introduce some basic concepts and then formal definition of learned index for exact similarity search in metric spaces is presented. Table 1 lists the key notations and acronyms used in this paper.

Definition 1 (Metric space). A metric space is a pair $(\mathbb{U}, dist)$, where $\mathbb{U}$ is a set of objects and $dist : \mathbb{U} \times \mathbb{U} \rightarrow [0, \infty)$ is a function so that $\forall p_1, p_2, p_3 \in \mathbb{U}$, the following holds:

- non-negativity: $dist(p_1, p_2) \geq 0$;
- identity: $dist(p_1, p_2) = 0$ iff $p_1 = p_2$;
- symmetry: $dist(p_1, p_2) = dist(p_2, p_1)$;
- triangle inequality: $dist(p_1, p_3) \leq dist(p_1, p_2) + dist(p_2, p_3)$.

Remark 1. Metric space is a generic space that can accommodate any type of data, as long as the associated distance metric satisfies the above properties. The vector space $\mathbb{R}^d$ with the L_p -norm ($p \geq 1$) is a special metric space.

In this paper, we consider three types of exact similarity queries in generic metric spaces: point query, range query and k NN query.

Definition 2 (Range query). Given a set $P \subseteq \mathbb{U}$, a query object $q \in \mathbb{U}$, and a query radius $r \in \mathbb{R}^*$, a range query returns all objects in P within distance r of q , i.e., $range(q, r) = \{p \in P | dist(p, q) \leq r\}$.

Definition 3 (Point query). Given a set $P \subseteq \mathbb{U}$ and a query object $q \in \mathbb{U}$, a point query returns all identical objects in P , i.e., $point(q) = \{p \in P | p = q\}$. Point query can be considered as a range query with a query radius $r = 0$ and an arbitrary metric.

Definition 4 (k NN query). Given a set $P \subseteq \mathbb{U}$, a query object $q \in \mathbb{U}$, and an integer $k \in \mathbb{N}^*$, a k NN query returns a set with

k objects, denoted as $kNN(q, k)$, such that $\forall p \in kNN(q, k), p' \in P \setminus kNN(q, k), dist(q, p) \leq dist(q, p')$.

Example 1. Consider a word dataset $P = \{\text{"fame"}, \text{"gain"}, \text{"aim"}, \text{"ACM"}\}$ associated with edit (Levenshtein) distance [11]. Range query $range(\text{"game"}, 2)$ returns all words in P within edit distance 2 to "game", i.e., $\{\text{"fame"}, \text{"gain"}\}$. 1NN query $kNN(\text{"game"}, 1)$ returns the nearest neighbor of "game", i.e., $\{\text{"fame"}\}$.

PROBLEM STATEMENT. Let $(\mathbb{U}, dist)$ be a metric space and $P = \{p_1, p_2, \dots, p_n\} \subseteq \mathbb{U}$ be a set of objects. The **learned index for exact similarity search in metric spaces** is to learn an index structure for P so that point query, range query and k NN query can be processed efficiently and accurately. In addition, the index structure is supposed to support insertion and deletion operations.

4 LIMS

In this section, we first present the main challenges of the problem and then describe in detail how LIMS solves them step by step.

4.1 Design Challenges

Learning an efficient metric access method for exact similarity search is a challenging journey.

- The first challenge is how to process massive data such that complex data distribution can be approximated by only a few simple models. Traditional indexes typically compose data objects into tree structures to represent part-whole hierarchies. Similarly, state-of-the-art learned indexes use a hierarchy of machine learning models to approximate global distribution. However, a complicated model incurs high query cost, even worse than linear scan.
- The second challenge is how to apply the learned index to metric spaces. A generic metric space has neither natural ordering nor coordinate structure and dimension information, which leads to the fact that numbering methods designed for vector spaces (e.g., z-order) are not applicable. Even for vector spaces, most learned indexes are also not fit because the grid partitioning strategy makes query performance deteriorate rapidly with increased dimensionality.
- The third challenge is how to minimize data objects accesses without significantly increasing pruning cost. In particular, due to the generality of the metric space, the triangle inequality is the only property we can utilize to reduce the search space during query processing.
- The fourth challenge is how to achieve efficient updates. Insertion and deletion affect the layout of the learned index and hence hurt the query performance. To maintain high query efficiency, the index must be rebuilt periodically. However, most existing learned indexes suffer from long training time, making them ill-adapted to frequent update and delete operations, and query pattern changes.

In what follows, we begin with data clustering and pivot selection (Section 4.2), which partially addresses the first and second challenges. Then, a novel index structure is developed to realize the second and fourth challenges (Section 4.3). Finally, we present LIMS-based query processing to completely settle exact similarity search in metric spaces (Section 5).

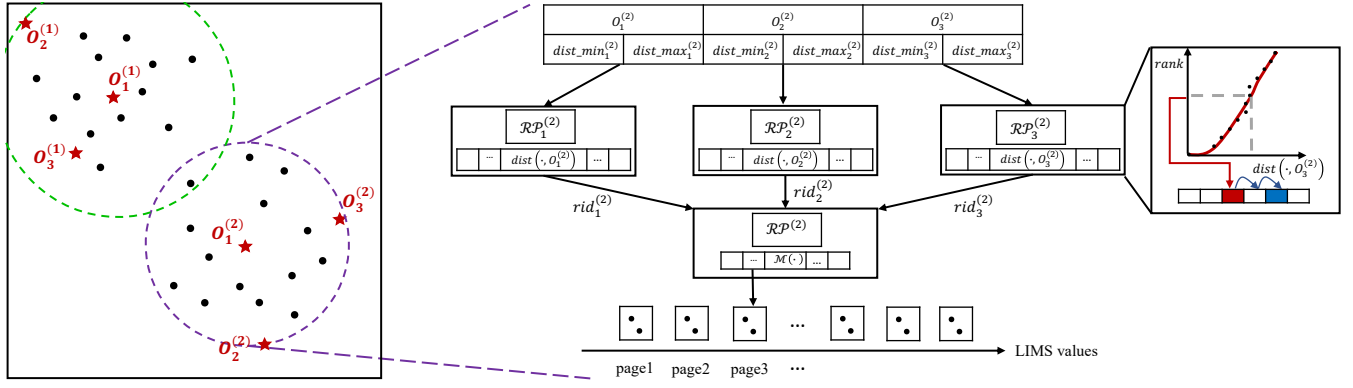


Figure 1: LIMS index structure

4.2 Data Clustering and Pivot Selection

Real-life data are usually clustered and correlated. Even when a global correlation does not exist, there may exist subsets of data locally correlated [3]. Inspired by this, LIMS adopts k-center algorithm [14, 16, 35], a simple yet effective algorithm that guarantees to return a 2-approximate optimal centroid set, to decompose complex data distribution into several simple distributions. To understand the role of data clustering, consider for example a dataset which represents points on a curved surface. When zooming into a small region of the surface, we can approximate it with a plane. This strategy gives two advantages: first, the data distribution of each cluster is more uniform, which simplifies the model to be learned (e.g., linear function); second, simple models have lower query and (re-)construction cost. The correct choice of the number of clusters is always ambiguous, with interpretations depending on many factors, such as data distribution and query workload. Therefore, we propose a statistic to determine the number of clusters that will be discussed in detail in Section 5.4. For now, we assume that the number of clusters has been determined.

Once the clusters are obtained, LIMS picks a few but effective pivots for each cluster. This is based on the observation that metric search performance depends critically on the intrinsic dimension, a property relying on data distribution itself, as opposed to the dimension where the data is represented [4, 8, 23]. Reconsider the previous example, the intrinsic dimension of a plane is two no matter if it is embedded in a high dimensional space. Therefore, re-distributing data with reference to well-chosen pivots may effectively ease the curse of dimensionality. While our metric-based indexing is not dependent on the underlying pivot selection method, the number and locations of pivots have an influence on retrieval performance. For one thing, more pivots provide more information and higher pruning power for query processing, but at the same time, time taken for checking pruning conditions increases (For LIMS, it mainly refers to the cost of generating search intervals that will be discussed in detail in Section 5). For another, a good pivots set can bound data objects more tightly and thus prune more clusters directly during query processing. Fortunately, methods for pre-defining an optimal set of pivots have been studied extensively [2, 5, 14, 20, 24, 32, 33]. In our implementation, we adopt farthest-first-traversal (FFT) algorithm [14] because of its linear time and space complexity. We

expect the number of clusters and pivots to be tuning parameters, which may vary for different datasets.

The above strategies provide an opportunity to design a new high performance index for high-dimensional metric data. On the basis of this, we proceed to present the details of index structure. Figure 1 gives an overview of LIMS in a metric space associated with L_2 -norm, although other metric spaces also apply for LIMS.

4.3 Index Structure

Suppose that K clusters, say $\{C_1, C_2, \dots, C_K\}$, and m pivots for each cluster, say $\{O_1^{(i)}, O_2^{(i)}, \dots, O_m^{(i)}\}$, $i \in \{1, \dots, K\}$, have been determined following steps in Section 4.2. Then, for each cluster C_i , $i \in \{1, \dots, K\}$ and pivot $O_j^{(i)}$, $j \in \{1, \dots, m\}$, all data objects in the cluster is sorted in ascending order with respect to their distances to the pivot. Maximum and minimum distances are also maintained in order to support efficient queries. Based on these sorted lists, LIMS divides the data space into N super ‘rings’ such that the data is covered as evenly as possible by each ‘ring’. Figure 2 is an example in a metric space associated with L_2 -norm, where the data objects in the cluster is partitioned into 3 rings with respect to pivot $O_1^{(1)}$ (Figure 2 (a)), and with respect to pivots $O_1^{(1)}$ and $O_3^{(1)}$ (Figure 2 (b)).

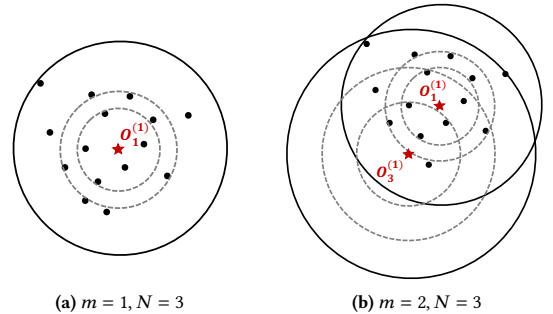


Figure 2: An example of data partitioning

In order to quickly find the ring ID where the data object is located, LIMS learns a set of *rank prediction models*. For model reuse, we define it formally as follows:

Definition 5 (Rank). Let A be a finite multiset drawn from an ordered set B . For any element $x \in A$, we define the rank of x as the number of elements smaller than x , i.e.,

$$\text{rank}(x) = |\{x' \in A \mid x' < x\}|. \quad (1)$$

Example 2. Let $A = \{1.5, 1.5, 1.8, 1.8, 2.0\}$ be a multiset of real numbers with usual order. Then, $\text{rank}(1.5) = 0$, $\text{rank}(1.8) = 2$, $\text{rank}(2.0) = 4$.

Definition 6 (Rank Prediction Model). Let A be a finite multiset drawn from an ordered set B . Rank prediction model $\mathcal{RP} : B \rightarrow [0, +\infty)$ is a function learned from $\{(x, \text{rank}(x))\}_{x \in A}$, so that it can predict the rank for any element $x \in B$, i.e.,

$$\text{rank}(x) \approx \mathcal{RP}(x). \quad (2)$$

Remark 2. In the strict sense, there is no definition of rank for element $x \in B \setminus A$. What we want to express here is the number of elements in A smaller than x . Without confusion, we still use "rank" for simplicity.

Let $D_j^{(i)} = \{\text{dist}(p, O_j^{(i)})\}_{p \in C_i}$ and $\tilde{D}_j^{(i)} = \{(x, \text{rank}(x))\}_{x \in D_j^{(i)}}$, then a series of rank prediction models $\mathcal{RP}_j^{(i)}$ can be learned based on $\tilde{D}_j^{(i)}$. $\mathcal{RP}_j^{(i)}$ is a polynomial function of x . For example, if the degree of polynomial is 2, then $\mathcal{RP}_j^{(i)}(x) = ax^2 + bx + c$, where a, b, c are parameters to be learned. The loss function $\mathcal{L}$ is defined as:

$$\mathcal{L} = \sum_{\tilde{D}_j^{(i)}} \left(\mathcal{RP}_j^{(i)}(x) - \text{rank}(x) \right)^2. \quad (3)$$

This is where the first learned index structure comes in. After $\mathcal{RP}_j^{(i)}$ is trained, we can get approximate rank for any object $p \in \mathbb{U}$ with distance in $D_j^{(i)}$, and the error can be easily corrected by exponential search in time $O(\log \text{err})$, where err is the difference between the estimated rank and the correct rank. Then, the ring ID, denoted as $\text{rid}_j^{(i)}$, is computed as:

$$\text{rid}_j^{(i)}(p) = \left\lfloor \frac{\text{rank}(\text{dist}(p, O_j^{(i)}))}{\lceil |D_j^{(i)}|/N \rceil} \right\rfloor. \quad (4)$$

For object $p \in \mathbb{U}$ with distance not in $D_j^{(i)}$, we can also use $\mathcal{RP}_j^{(i)}$ and exponential search to find its corresponding rank, i.e., the rank of the first element larger than $\text{dist}(p, O_j^{(i)})$ in $D_j^{(i)}$, with the same time complexity.

When the above two steps are completed, each data object is equipped with m ring IDs. Then, LIMS designs a novel *pivot-based mapping function* $\mathcal{M}$ to transform all data objects in the metric space into ordered sets with an associated binary relation $\leq$. The formal definitions are as follows.

Definition 7 (Pivot-based mapping function). Given a cluster C_i , $i \in \{1, \dots, K\}$, and its corresponding pivots $O_j^{(i)}$, $j \in \{1, \dots, m\}$, let $\{\text{rid}_1^{(i)}, \dots, \text{rid}_m^{(i)}\}$ be a set of m ring ID functions. Then, for any data object $p \in \mathbb{U}$, we define a pivot-based mapping function $\mathcal{M}$ as follows:

$$\mathcal{M}(p) = \text{rid}_1^{(i)}(p) \oplus \text{rid}_2^{(i)}(p) \oplus \dots \oplus \text{rid}_m^{(i)}(p). \quad (5)$$

We call $\mathcal{M}(p)$ LIMS value of p .

Note that $\oplus$ is a generic binary operator. Here, we do not provide a specific definition for $\oplus$. Instead, we define a binary relationship $\leq$ between LIMS values as Definition 8, which is adequate to build a learned index structure.

Example 3. Consider a cluster consists of 4 data objects p_1, p_2, p_3, p_4 . Suppose $m = 2$ and $N = 4$. Then, objects are sorted in ascending order of their distances to each pivot, and are assigned into 4 super rings. Say, $\text{rid}_1(p_4) = 0, \text{rid}_1(p_1) = 1, \text{rid}_1(p_2) = 2, \text{rid}_1(p_3) = 3$ and $\text{rid}_2(p_3) = 0, \text{rid}_2(p_2) = 1, \text{rid}_2(p_1) = 2, \text{rid}_2(p_4) = 3$, respectively. According to the definition of pivot-based mapping function, $\mathcal{M}(p_1) = 1 \oplus 2, \mathcal{M}(p_2) = 2 \oplus 1, \mathcal{M}(p_3) = 3 \oplus 0$ and $\mathcal{M}(p_4) = 0 \oplus 3$.

Definition 8 (Binary Relation $\leq$). Let S be a multiset of LIMS values in cluster C_i with m pivots, $i \in \{1, \dots, K\}$. Then, S can be ordered as follows:

$$\text{rid}_1^{(i)}(p) \oplus \dots \oplus \text{rid}_m^{(i)}(p) \leq \text{rid}_1^{(i)}(p') \oplus \dots \oplus \text{rid}_m^{(i)}(p') \quad (6)$$

if and only if condition (1) or (2) is satisfied.

$$(1) \forall j \in \{1, \dots, m\}$$

$$\text{rid}_j^{(i)}(p) = \text{rid}_j^{(i)}(p'); \quad (7)$$

$$(2) \exists k \in \{1, \dots, m\} \text{ such that}$$

$$\text{rid}_j^{(i)}(p) = \text{rid}_j^{(i)}(p') \text{ for } j < k \text{ and } \text{rid}_k^{(i)}(p) < \text{rid}_k^{(i)}(p'), \quad (8)$$

where " $=$ " and " $<$ " in conditions is the order of natural numbers.

It's straightforward to prove binary relation $\leq$ is well-defined, i.e., it is reflexive, antisymmetric and transitive [29], and $(S, \leq)$ is an ordered set. By pivots-based mapping function, similar objects in metric space are mapped into compact region in the ordered set. Proof is omitted.

Example 4. In the above example, four LIMS values are sorted as follows: $\mathcal{M}(p_4) < \mathcal{M}(p_1) < \mathcal{M}(p_2) < \mathcal{M}(p_3)$.

Now that all objects in the metric space are transformed into corresponding ordered sets, we can sort them sequentially in ascending order of LIMS values, and store data in a number of pages with each page fully utilized. To quickly locate addresses of data objects, LIMS learns a *rank prediction model*. Specifically, let $D^{(i)} = \{\mathcal{M}(p)\}_{p \in C_i}$ be a multiset of LIMS values of $p \in C_i$ and $\tilde{D}^{(i)} = \{(x, \text{rank}(x))\}_{x \in D^{(i)}}$, then a rank prediction model $\mathcal{RP}^{(i)}$ can be learned based on $\tilde{D}^{(i)}$. Similar to the loss function Equation 3, we still try to minimize squared error. This is where the second learned index structure comes in. After $\mathcal{RP}^{(i)}$ is trained, we can get approximate rank for any object $p \in \mathbb{U}$ with LIMS value $\mathcal{M}(p) \in D^{(i)}$. The error can be easily corrected by exponential search that stops when the first occurrence of $\mathcal{M}(p)$ is found. For object $p \in \mathbb{U}$ with LIMS value not in $D^{(i)}$, we can also use $\mathcal{RP}^{(i)}$ and exponential search to find its corresponding rank, i.e., the position where the the first occurrence of the element larger than $\mathcal{M}(p)$ in $D^{(i)}$.

LIMS abandons longstanding tree-like index structure, instead maintains a learned index for each cluster separately. Such organization schema allows the index can be partially reconstructed quickly, but also brings a problem of how to achieve efficient and accurate similarity queries, which will be discussed further below.

5 LIMS-BASED QUERY PROCESSING

In this section, we present query processing algorithms for point, range and k NN queries using LIMS. Without loss of generality, we assume there is no two objects in P are totally same.

5.1 Range Query

Given a query object q , query radius r and dataset P , range query is to retrieve all objects in P within distance r of q , i.e., $\text{range}(q, r) = \{p \in P | \text{dist}(p, q) \leq r\}$. Algorithm 1 outlines the range query processing, which consists of 4 subprocedures: *TriPrune* (Line 1-6), prune irrelevant clusters by triangle inequality; *AreaLocate* (Line 6-18), determine affected areas of relevant clusters with the help of rank prediction functions $\mathcal{RP}_j^{(i)}$; *IntervalGen* (Line 19), generate search intervals on LIMS values; *PosLocate* (Line 20-31), locate positions of records in the disk by rank prediction functions $\mathcal{RP}^{(i)}$. **TriPrune** (Line 1-6). The algorithm starts by computing the distances between the query q and the pivots, and then utilizes triangle inequality property in the metric space to prune a number of irrelevant clusters and thus accelerate the search. Specifically, according to triangle inequality, an object p in cluster C_i may fall into the query range, it must satisfy

$$\text{dist}(O_j^{(i)}, q) - r \leq \text{dist}(O_j^{(i)}, p) \leq \text{dist}(O_j^{(i)}, q) + r; j = 1, \dots, m. \quad (9)$$

Recall that LIMS also maintains both maximum distance $\text{dist_max}_j^{(i)}$ and minimum distance $\text{dist_min}_j^{(i)}$ of the cluster, hence, for any object p in the cluster, we must have:

$$\text{dist_min}_j^{(i)} \leq \text{dist}(O_j^{(i)}, p) \leq \text{dist_max}_j^{(i)}; j = 1, \dots, m. \quad (10)$$

Combining Equation 9 and Equation 10, we can derive that a cluster $C_i, i \in \{1, \dots, K\}$ is needed to be further checked if and only if the following condition (Line 4) is satisfied.

$$\bigwedge_{j=1}^m \text{dist}(O_j^{(i)}, q) \leq \text{dist_max}_j^{(i)} + r \wedge \text{dist}(O_j^{(i)}, q) \geq \text{dist_min}_j^{(i)} - r. \quad (11)$$

AreaLocate (Line 6-18). For a cluster C_i that needs to be searched, LIMS first determines the affected areas in the metric space according to the following equations:

$$r_min_j^{(i)} = \max\{\text{dist}(O_j^{(i)}, q) - r, \text{dist_min}_j^{(i)}\}; j = 1, \dots, m; \quad (12)$$

$$r_max_j^{(i)} = \min\{\text{dist}(O_j^{(i)}, q) + r, \text{dist_max}_j^{(i)}\}; j = 1, \dots, m. \quad (13)$$

Then, LIMS invokes corresponding rank prediction models $\mathcal{RP}_j^{(i)}$ to predict the min and max *rank*s of affected areas and the error is fixed via exponential search (Line 11-13). Min ring ID can be easily calculated by calling $\text{rid}_j^{(i)}$ function, while the max ring ID is divided to 2 cases in order to narrow down search range as much as possible (Line 14-18).

IntervalGen (Line 19). Instead of doing intersection of several candidate sets in the metric space via costly distance computations, LIMS reduces search space by doing intersection of LIMS-value intervals directly. Specifically, for a relevant cluster $C_i, i \in$

Algorithm 1: Range Query

Input: q : a query object; r : query radius
Output: S : objects in P satisfying $\text{dist}(p, q) \leq r$

- 1 Let $\text{flag}[1..K]$ be an array of all *TRUE*, $\text{rid_min}[1..m]$, $\text{rid_max}[1..m]$ be arrays of all 0;
- 2 **for each** cluster C_i **do**
- 3 **for each** pivot $O_j^{(i)}$ **do**
- 4 **if** $\text{dist}(O_j^{(i)}, q) > \text{dist_max}_j^{(i)} + r$ **OR**
 $\text{dist}(O_j^{(i)}, q) < \text{dist_min}_j^{(i)} - r$ **then**
- 5 $\text{flag}[i] = \text{FALSE}$;
- 6 **break**;
- 7 **for each** *TRUE* cluster C_i **do**
- 8 **for each** pivot $O_j^{(i)}$ **do**
- 9 $r_min \leftarrow \max\{\text{dist}(O_j^{(i)}, q) - r, \text{dist_min}_j^{(i)}\}$;
- 10 $r_max \leftarrow \min\{\text{dist}(O_j^{(i)}, q) + r, \text{dist_max}_j^{(i)}\}$;
- 11 $\text{rank}'_{min}, \text{rank}'_{max} \leftarrow \mathcal{RP}_j^{(i)}(r_min), \mathcal{RP}_j^{(i)}(r_max)$;
- 12 $\text{rank}_{min} \leftarrow \text{ExponentialSearch}(\text{rank}'_{min}, r_min)$;
- 13 $\text{rank}_{max} \leftarrow \text{ExponentialSearch}(\text{rank}'_{max}, r_max)$;
- 14 /*assume $\text{rank}_{min} < \text{rank}_{max}$, otherwise discard C_i */;
- 15 $\text{rid_min}[i] \leftarrow \text{rid}^{(i)}(\text{rank}_{min})$;
- 16 **if** $D_j^{(i)}[\text{rank}_{max}] = r_max$ **then**
- 17 $\text{rid_max}[i] \leftarrow \text{rid}^{(i)}(\text{rank}_{max})$;
- 18 **else**
- 19 $\text{rid_max}[i] \leftarrow \text{rid}^{(i)}(\text{rank}_{max} - 1)$;
- 20 generate LIMS-value ranges $\mathcal{R}$ based on $\text{rid_min}, \text{rid_max}$;
- 21 **for each** range $I \in \mathcal{R}$ **do**
- 22 $\text{lbound}', \text{ubound}' \leftarrow \mathcal{RP}^{(i)}(I.\text{left}), \mathcal{RP}^{(i)}(I.\text{right})$;
- 23 $\text{lbound} \leftarrow \text{ExponentialSearch}(\text{lbound}', I.\text{left})$;
- 24 $\text{ubound} \leftarrow \text{ExponentialSearch}(\text{ubound}', I.\text{right})$;
- 25 **if** $D^{(i)}[\text{ubound}] = I.\text{right}$ **then**
- 26 $\text{ubound} \leftarrow \text{ExponentialSearch2}(\text{ubound}, I.\text{right})$;
- 27 **else**
- 28 $\text{ubound} \leftarrow \text{ubound} - 1$;
- 29 /*assume $\text{lbound} < \text{ubound}$, otherwise discard I */;
- 30 add to $\mathcal{P}$ all unvisited pages from $\lfloor \text{lbound}/\Omega \rfloor$ to $\lfloor \text{ubound}/\Omega \rfloor$;
- 31 add to S all objects saved in pages $\mathcal{P}$ satisfying $\text{dist}(p, q) \leq r$;
- 32 **return** S

$\{1, \dots, K\}$, let $L_j^{(i)} = \{\text{rid_min}_j^{(i)}, \text{rid_min}_j^{(i)} + 1, \dots, \text{rid_max}_j^{(i)}\}$, $j \in \{1, \dots, m-1\}$ and $L_m^{(i)} = \{\text{rid_min}_m^{(i)}, \text{rid_max}_m^{(i)}\}$. Deep first search (DFS) is run on a directed acyclic graph (DAG) composed of vertexes $\cup_{j=1}^m L_j^{(i)}$ and fully connected edges from $L_j^{(i)}$ to $L_{j+1}^{(i)}$, $j \in \{1, \dots, m-1\}$, to find all paths from $L_1^{(i)}$ to $L_m^{(i)}$. These paths form a total of $\prod_{j=1}^{m-1} |L_j^{(i)}|$ LIMS-value search ranges. Following is an example of this step.

Example 5. Consider a cluster with $m = 3$ pivots and $N = 10$ rings. Suppose it is relevant to a range query such that the minimum and maximum ring IDs of affected region w.r.t. the 1st, 2nd and 3rd pivots are $\text{rid_min}_1 = 2, \text{rid_max}_1 = 4, \text{rid_min}_2 = 6, \text{rid_max}_2 = 8, \text{rid_min}_3 = 1, \text{rid_max}_3 = 5$, respectively, i.e., $L_1 = \{2, 3, 4\}, L_2 =$

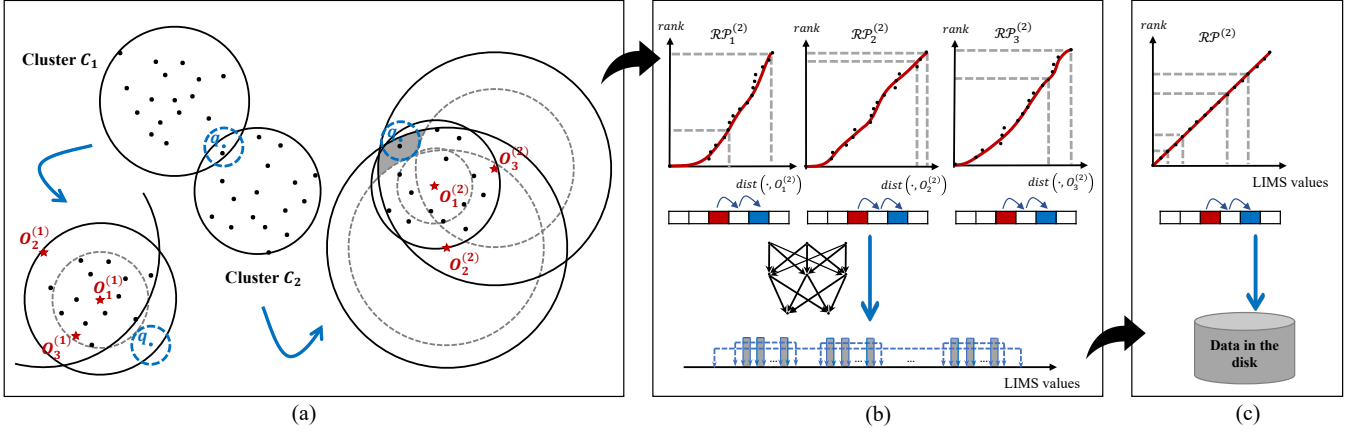


Figure 3: An example of range query based on LIMS

$\{6, 7, 8\}$ and $L_3 = \{1, 5\}$. Then, LIMS-value search ranges can be computed by running DFS on the DAG shown in Figure 4. The final search ranges are the union of $[2 \oplus 6 \oplus 1, 2 \oplus 6 \oplus 5]$, $[2 \oplus 7 \oplus 1, 2 \oplus 7 \oplus 5]$, $[2 \oplus 8 \oplus 1, 2 \oplus 8 \oplus 5]$, $[3 \oplus 6 \oplus 1, 3 \oplus 6 \oplus 5]$, $[3 \oplus 7 \oplus 1, 3 \oplus 7 \oplus 5]$, $[3 \oplus 8 \oplus 1, 3 \oplus 8 \oplus 5]$, $[4 \oplus 6 \oplus 1, 4 \oplus 6 \oplus 5]$, $[4 \oplus 7 \oplus 1, 4 \oplus 7 \oplus 5]$ and $[4 \oplus 8 \oplus 1, 4 \oplus 8 \oplus 5]$.

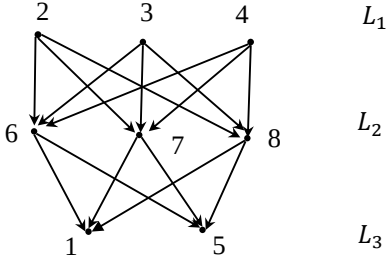


Figure 4: An illustration of finding LIMS-value search ranges

Note that the intersection contains all positive objects in the result set, although it may include negative objects (i.e., false hits). So a final refinement step must be applied to ensure the correctness of queries. The exact distance computations between candidates and the query object are performed only in the refinement step. It is through *IntervalGen* that the number of data objects to be accessed is significantly reduced, while the pruning cost remains low.

PosLocate (Line 20-31). For each search range, the position of lower bound in the disk can be easily located via rank prediction model $\mathcal{RP}^{(i)}$ and exponential search. Ω is the maximum number of objects each page can hold. The upper bound is divided into 2 cases to make sure the results correct. For example, it's possible that different objects have the same LIMS value, so we find the last occurrence of the LIMS value via another exponential search, denoted as *ExponentialSearch2*, to guarantee no objects missed (i.e., no false negatives). Finally, all retrieved objects are further refined in the refinement step (Line 30), where exact distance computations are performed.

Query Cost. The first subprocedure, *TriPrune*, takes $O(mKD)$ time, where D represents the cost of distance computation. The second

cost component, *AreaLocate*, depends on rank prediction models. We use $O(\mathcal{RP})$ and $O(\log \text{err})$ to denote the prediction cost of $\mathcal{RP}_j^{(i)}$ or $\mathcal{RP}^{(i)}$, and the cost of fixing error incurred by models via exponential search, respectively. Hence, we need $O(mK(\mathcal{RP} + \log \text{err}))$ time to locate affected areas of relevant clusters. The cost of *IntervalGen* is from running DFS on DAG, which takes $O(|V| + |E|)$ time, where $|V|$ is the number of vertexes and $|E|$ is the number of edges. The fourth subprocedure, *PosLocate*, takes $O(|\mathcal{R}|(\mathcal{RP} + \log \text{err}))$ time, where $|\mathcal{R}|$ is the number of LIMS-value search ranges. Generally, $|\mathcal{R}| > |E| > |V|$. In addition, we need to access disk pages in $\mathcal{P}$ to refine and retrieve final result, which takes $O(|\mathcal{P}|\Omega D)$ time. Therefore, the overall query time is $O((mK + |\mathcal{R}|)(\mathcal{RP} + \log \text{err}) + (mK + |\mathcal{P}|\Omega)D)$.

Figure 3 is an example of LIMS-based range query. It's straightforward to see that cluster C_1 does not satisfy Equation 11, and thus can be discarded from further processing directly, while cluster C_2 cannot be discarded. As for cluster C_2 , LIMS first inputs the min and max boundaries (i.e., grey dashed lines in Figure 3 (a)) of affected areas into corresponding rank prediction models $\mathcal{RP}_j^{(2)}$ and ring ID functions $\text{rid}_j^{(2)}$, $j = 1, 2, 3$. Then, LIMS runs DFS on DAG to transform the intersection of several candidate sets in the metric space (i.e., grey region in 3 (a)) to the intersection of intervals (i.e., grey region in Figure 3 (b)), to significantly reduces the number of distance computations and page accesses. Next, positions of objects in the disk are estimated by the rank prediction function $\mathcal{RP}^{(2)}$. Finally, candidate objects are retrieved and a refinement step is applied (Figure 3 (c)).

Point query can be easily achieved by treating it as a simplified range query with infinitesimal query radius and an arbitrary metric. To further boost performance, we can make use of the property of k-center algorithm that an object q belongs to a cluster with the nearest centroid, to prune $K - 1$ clusters directly. Then, we can invoke $\mathcal{RP}_j^{(i)}$ and exponential search to locate rank of q , and the search stops once $\text{dist}(q, O_j^{(i)})$ not in $D_j^{(i)}$ is found. If $q \in \mathcal{P}$, remaining query procedure is similar to range query, so we omit pseudocode here due to space constraints.

5.2 *k*NN Query

In LIMS, *k*NN query is achieved by a series of range queries with increasing radius, *i.e.*, it begins by searching a small region around the query object q and keeps enlarging the search space until k nearest neighbors are found. Algorithm 2 outlines the LIMS-based *k*NN query. The search starts with a given initial radius Δr . Each time the search region is expanded, the radius increases by Δr (Line 4) and a range query $range(q, r)$ is run using Algorithm 1 (Line 7). Q is a max priority queue to record the candidate k nearest neighbors (Line 2). If the distance from retrieved object p to the query object q is smaller than the current furthest distance in Q , LIMS will extract the object with the maximum distance value from Q and insert p into Q (Line 12-15). LIMS also maintains an array to record whether a page has been processed. If so, it will be skipped during the next range query to avoid repeated accesses (Line 10). In order to guarantee the correctness of *k*NN query, the search stops if and only if the furthest object in the current priority queue falls within the current query range and further expansion of the query radius does not change the answer set (Line 5-9).

5.3 Updates

LIMS adopts an easy but effective method, *i.e.*, maintaining a sorted array for each cluster, to support insertion of data. Specifically, given a new object p to be inserted, LIMS first calculates the distance between p and centroid of each cluster to find the closest. Then, LIMS inserts p into the array of this cluster based on the ascending order of distance value. All new inserted data are arranged in a number of pages with each page fully utilized. During query processing, LIMS uses triangle inequality and exponential searches to retrieve these inserted objects matching query filters. Note that exponential searches can be replaced by *rank prediction models*, but in this paper, we still adopt this easy strategy for three reasons. First, LIMS partitions the data space into many clusters which amortize the additional search time on the sorted arrays. Second, LIMS maintains an index for each cluster separately, enabling retraining part of index structure when the query performance degrades severely. Third, short index construction time of LIMS ensures the feasibility of this method in practice (Section 6). Given an object p to be deleted, LIMS first runs a point query to find if there is a page containing p . If so, this object is marked as ‘deleted’. Then, LIMS updates maximum and minimum distances to each pivot of the cluster p belongs to.

5.4 Last Piece

The last piece of LIMS is to determine the number of clusters. The optimal number is related not only to data distribution but also to query workload. However, query workload is not available during data clustering and we do not assume a known query workload in this paper. Therefore, an alternative should be developed to predefine the number of clustering parameter K . Recall that the goal of clustering in LIMS is to decompose complex and potentially correlated data into a few clusters. Ideally, these clusters are independent and each can be accurately fit by a linear function. However, in most real-world use cases, the data do not follow such perfect pattern. On the one hand, the overlapping between clusters may incur extra pruning and refinement cost. On the other hand, uneven intra-cluster distribution incurs more arithmetic and comparison

Algorithm 2: *k*NN Query

Input: q : a query point; k : a positive integer; Δr : a positive integer
Output: S : top k nearest neighbors to q

```

1  $r \leftarrow 0, flag \leftarrow FALSE$ ;
2  $Q$  is a max priority queue on  $k$  objects with  $\infty$  distance to  $q$ ;
3 while  $flag = FALSE$  do
4    $r \leftarrow r + \Delta r$ ;
5   if  $dist(Q[1], q) < r$  then
6      $flag \leftarrow TRUE$ ;
7   call  $range(q, r)$  to get pages set  $\mathcal{P}$ ;
8   call  $check(\mathcal{P})$ ;
9   return  $Q$ ;
10 procedure  $check(\mathcal{P})$ :
11 for each unvisited page  $P \in \mathcal{P}$  do
12   for each point  $p \in P$  do
13     if  $dist(Q[1], q) > dist(p, q)$  then
14       Extract-Max( $Q$ );
15       Insert( $Q, p$ );
```

operations. In order to pick a K to avoid or reduce such overhead, we introduce *overlap rate* (OR) and *mean absolute error* (MAE) to evaluate the goodness of clustering. OR quantifies the extent of overlap among clusters. For each $K \in \mathbb{N}^*$, it can be computed as:

$$OR = \frac{1}{K(K-1)} \sum_{i=1}^K \sum_{i' \neq i} \frac{r^{(i,i')}}{dist_max_1^{(i)}}. \quad (14)$$

Without confusion, we use $dist_max_1^{(i)}$ to represent the distance of the furthest object in i th cluster from the centroid for simplicity, and $r^{(i,i')}$ is the length of the overlap area computed as:

$$r^{(i,i')} = \min\{dist(O_1^{(i)}, O_1^{(i')}) + dist_max_1^{(i')}, dist_max_1^{(i)}\} - \max\{(dist(O_1^{(i)}, O_1^{(i')}) - dist_max_1^{(i')}, dist_min_1^{(i)}\}. \quad (15)$$

MAE quantifies the quality of the linear regression fit for each cluster. For each $K \in \mathbb{N}^*$, it can be computed as:

$$MAE = \frac{1}{m|P|} \sum_{i=1}^K \sum_{j=1}^m \sum_{\tilde{D}_j^{(i)}} |a_j^{(i)} x + b_j^{(i)} - rank(x)|, \quad (16)$$

where $|P|$ is the cardinality of the dataset, $\mathcal{RP}_j^{(i)}(x) = a_j^{(i)} x + b_j^{(i)}$ are a series of linear rank prediction models learned from $\tilde{D}_j^{(i)}$.

We model the overhead as $OR + \lambda MAE$, where $\lambda \in \mathbb{R}^*$ is a user-defined weight. Inspired by the elbow method [31], we choose the ‘elbow’ or ‘knee of a curve’ as the clustering number to use, *i.e.*, a point where adding more clusters will not give much better modeling of the data. The number estimated by the techniques described above turn out to be very close to the optimal number of clusters observed in practice, as we will show in the experimental section that follows.

6 EXPERIMENTS

In this section, we experimentally evaluate the performance of LIMS. First, we describe the experimental settings. Then, we study the effect of parameters for LIMS. Finally, we present the results of an in-depth experimental study that compares LIMS with several indexing methods from different angles on a variety of datasets.

6.1 Experimental Settings

We implement LIMS¹ and associated similarity search algorithms in C++. All experiments are conducted on a computer running 64-bit Ubuntu 20.04 with a 2.30 GHz Intel(R) Xeon(R) Gold 5218 CPU, 254 GB RAM, and a 8.2 TB hard disk.

Datasets. We employ two real-world datasets, namely, *Color Histogram*² and *Forest Cover Type*³ following the experimental settings of ML index [9]. *Color Histogram* contains 1,281,167 32-dimensional image features extracted from the ImageNet⁴ dataset. *Forest Cover Type* is collected by US Geological Survey and US Forest Service. It includes 565,892 records, each of which have 12 cartographic variables. We extract 6 quantitative variables of them as our data object. Following the experimental settings of iDistance [15], we generate 2, 4, 8, 12, 16-dimensional *GaussMix* datasets. Every dataset contains 10 million points sampled from 150 normal distributions with the default standard deviation of 0.05 and randomly determined means. Without loss of generality, L_2 norm is utilized to the above datasets. Following the experimental settings of RSMI [26], we create 2, 4, 8, 12, 16-dimensional *Skewed* datasets. They are generated from uniform data by raising the values in each dimension to their powers, i.e., a randomly generated data point are converted from $(x_1, x_2, \dots, x_d)$ to $(x_1, x_2^2, \dots, x_d^d)$. The size of each dataset is 10 million and L_1 norm is employed. Without loss of generality, all the data values of the above datasets are normalized to the range $[0, 1]$. Following the experimental settings in [30], we also generate a *Signature* dataset, where each object is a string with 65 English letters. We first obtain 25 ‘anchor signatures’ whose letters are randomly chosen from the alphabet. Then, each anchor produces a cluster with 4,000 objects, each of which is obtained by randomly changing x positions in the corresponding anchor signature to other random letters, where x is uniformly distributed in range $[1, 30]$. The *edit distance* is used to compute the distance between two signatures. Table 2 summarizes the statistics of the datasets.

Table 2: Summary of datasets

Datasets	Cardinality	Dim.	Distance metric
<i>Color Histogram</i>	1,281,167	32	L_2 -norm
<i>Forest Cover Type</i>	565,892	6	L_2 -norm
<i>GaussMix</i>	10,000,000	2,4,8,12,16	L_2 -norm
<i>Skewed</i>	10,000,000	2,4,8,12,16	L_1 -norm
<i>Signature</i>	100,000	65	Edit distance

Competitors. We compare LIMS with three representative multi-dimensional learned indexes as mentioned in section 2, i.e., ZM [34], ML [9], LISA [19], and two traditional indexes, i.e., R*-tree [1] and

M-tree [7]. ZM and ML are in-memory indexes, so we adapt them to the disk by storing data in ascending order of their Z-order/mapped values in a number of pages with each page fully utilized. For LISA, no open-source C++ code is available, so we implement it following Python version implementation. For R*-tree and M-tree, we use the original implementations. All competitors are configured to use a fixed disk page size of 4KB.

Evaluation Metric. Four metrics are used to evaluate the performance of the indexes: the average number of page accesses; the average query time; index construction time and index size. We randomly select 200 objects from each dataset and repeat each experiment 20 times to get average results.

6.2 Effect of Parameters

We first study the effect of parameters, including the number of clusters K , the number of pivots m and the number of rings N , to optimize LIMS-based similarity searches, as summarized in Figure 5. Note that, we only employ range query to demonstrate the impact of parameters due to similar performance behavior of k NN query, and only one parameter varies whereas the others are fixed to their default values in every experiment.

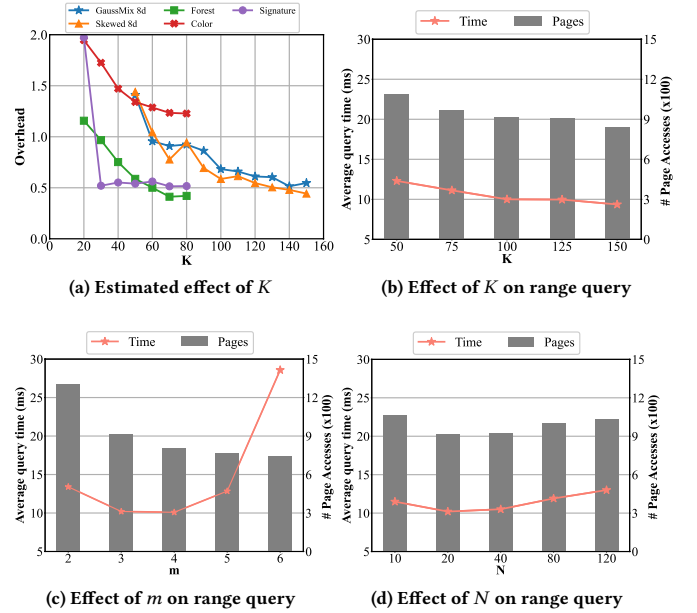


Figure 5: Effect of parameters

Effect of K . Figure 5 (a) plots the criterion $OR + \lambda MAE$ versus K on both real and synthetic data, where $K \in \{20, 30, \dots, 150\}$ and λ is set to $1/\max\{MAE(K)\}$. We can see that different datasets have different elbow points. In order to show the estimation is close to the actual optimal number of clusters, we also plot the actual average query time and the number of page accesses for range query by varying K . Due to the space limitation, we only report the performance on 8d *GaussMix* dataset in Figure 5 (b), where the selectivity, i.e., the fraction of objects within the query range from the total number of objects, is set to 0.01% (default). As Figure 5 (b) shows, query time decreasing slowly after $K = 100$,

¹<https://github.com/learned-index/LIMS>

²<https://image-net.org/download-images>

³<https://www.kaggle.com/c/forest-cover-type-prediction/data>

⁴<http://image-net.org/download-images>

which is consistent with the recommended choice of K in Figure 5 (a). Therefore, we set $K = 100$ for *8d GaussMix* dataset.

Effect of m . Figure 5 (c) reports the query performance on *8d GaussMix* dataset by varying the number of pivots m . We can see that increasing the number of pivots always reduces (or at least does not increase) the number of page accesses. This is expected because the intersection of metric regions defined by more pivots is always smaller than (or at least equal to) the intersection of metric regions defined by fewer pivots. As discussed in section 4.2, the more pivots, the stronger pruning ability. This observation can also be proven using Equation 11. However, average query time decreases when using up to four pivots, and then increases progressively. That's because the cost for filtering unqualified objects also grows as well with more pivots (For LIMS, it mainly refers to cost for finding search intervals). The best number of pivots for a metric index is a trade-off between filter cost and scanning cost. Therefore, default value for m is set to 3 unless otherwise stated.

Effect of N . Figure 5 (d) reports the query performance on *8d GaussMix* dataset by varying the number of rings N . For the same reason as above, the average query time presents a down and up trend whereas the lowest value turns out at $N = 20$. Therefore, we set $N = 20$ unless otherwise stated.

6.3 Range Query Performance

In this subsection, we study the performance of LIMS, ML, LISA, ZM, R^* -tree and M-tree on range query from different angles, as summarized in Figure 6, 7 and 10.

Performance with dimensionality. The first set of experiments studies the average query time and the number of page accesses under different dimensionalities. The selectivity is fixed to 0.01%. Figure 6 (a)(b) report the results on the *Skewed* datasets. LIMS is slightly slower than LISA when the dimension is 2, but it keeps performing better than all competing techniques since *4d*. This is because grid-based indexing can easily locate the cells that overlap with the query range, while a search through multiple clusters requires more time. In a higher range of dimension, LIMS offers the best performance in terms of both query time and page accesses, mainly due to two reasons. First, LIMS does not need coordinate information when processing queries, which results in a robust capacity to deal with similarity searches under various distance metrics (e.g., L_1 norm), and hence fewer false hits and faster query response. Second, LIMS uses clusters and LIMS values to organize objects into compact regions, leading to fewer page accesses. In addition, due to clustering partitioning strategy, LIMS can well adapt to skewed data distribution and outperforms ZM by over an order of magnitude even in 2 dimensional case. We don't report results of LISA, ZM and R^* -tree when the dimensionality grows to 12 because they are considerably slow.

Figure 6 (c)(d) report the results on the *GaussMix* datasets. Due to the essence of simple distribution, all indexes achieve better performance, but LIMS still has a lead in average query time except *2d* and *4d* where LISA is relatively faster (0.94ms v.s. 0.53ms and 3.2ms v.s. 2.4ms). Note that even though LIMS has higher numbers of page accesses than LISA in *8d*, it's still the fastest due to simple checking procedure. Specifically, more page accesses of LIMS is because generic metric indexes can only use the four distance properties to

prune the search space. Less information about the data may result in poorer pruning. However, filtering cost of grid-based indexing (LISA) grows exponentially with the dimensionality, such that it does not work after *8d*. The poor performance of ZM and R^* -tree is mainly because too many false hits and large minimum bounding box, respectively. ML has similar tendency but not as good as LIMS. This is intuitive since ML transforms different objects with equi-distance from the pivot into the same 1-dimensional value, while LIMS integrates the pruning abilities from multiple pivots. In addition, with the help of a well-defined pivots-based mapping, LIMS maps nearby data into compact region, which further reduces the search region. Therefore, fewer objects have to be accessed for LIMS in the refinement step. Even though the advantages of LIMS in page accesses come with a higher filter cost to find search intervals, LIMS is still faster.

Performance with selectivity. The second set of experiments studies the average query time and the number of page accesses with different selectivity. Figure 7 (a)(b) show the results on the *Forest Cover Type (6d)* dataset by varying selectivity from 0.1% to 4%. K and N on this small dataset is cut in half. Both query time and page access among indexes grows with the selectivity since more points are queried. LIMS achieves better performance under all selectivities by at least a factor of 1.7 and up to 4.4 times. Consistent high efficiency of LIMS shows robustness for range queries in varying settings.

Figure 7 (c)(d) show the results on the *Color Histogram (32d)* dataset by varying the selectivity from 0.05% to 0.8%. K and N on this small dataset is cut in half. We exclude ZM, LISA and R^* -tree because they cannot be applied in such high dimension. Clearly, LIMS is always better than ML in both query time and page accesses because the reason we discusses above. Particularly, the advantage in fewer false hits shows the potential advantages LIMS can achieve over ML for processing similarity searches with costly metric distance measures.

Performance in generic metric spaces. The third set of experiments studies the average query time and the number of page accesses in a generic metric space. We only compare LIMS with M-tree because other competitors are not applicable for generic metric spaces. Figure 10 reports the results on the *Signature* dataset. K is set to 50. Clearly, LIMS has a decided advantage over M-tree under all selectivities, where LIMS is around 20X faster than M-tree and the page accesses is at least 12X fewer. This is expected because of expensive and unavoidable cost in traditional index structures, such as traversing tree structure and chasing pointers.

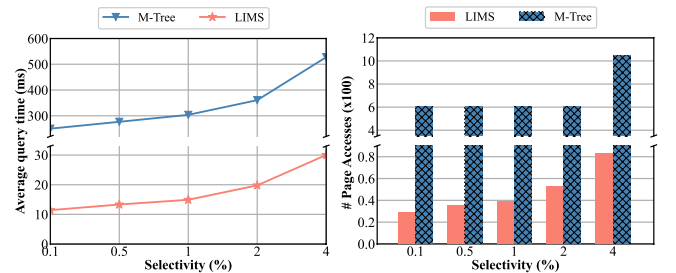


Figure 10: Range query performance on *Signature*

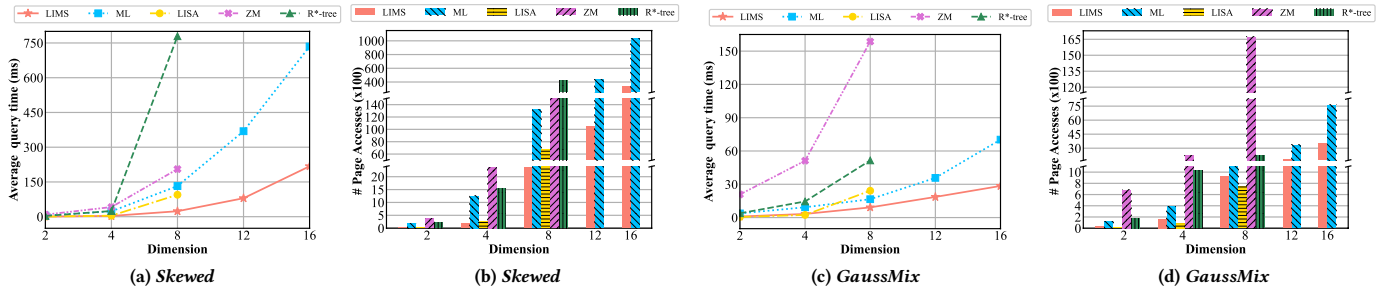


Figure 6: Range query performance with dimensionality

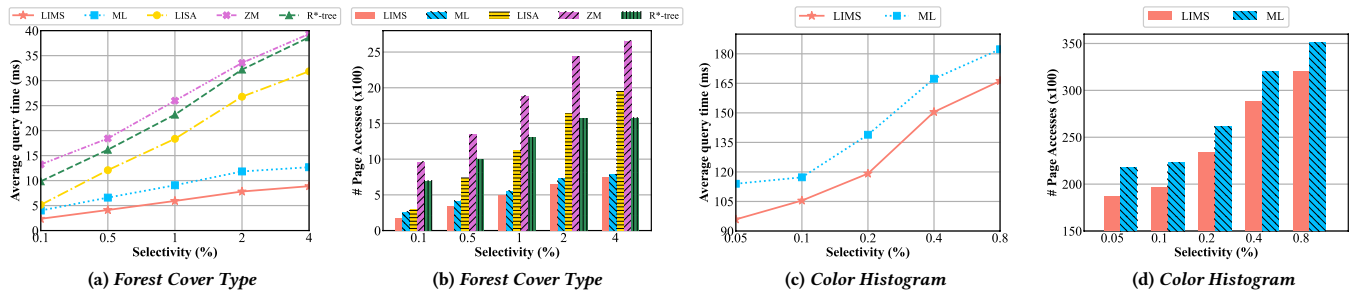


Figure 7: Range query performance with selectivity

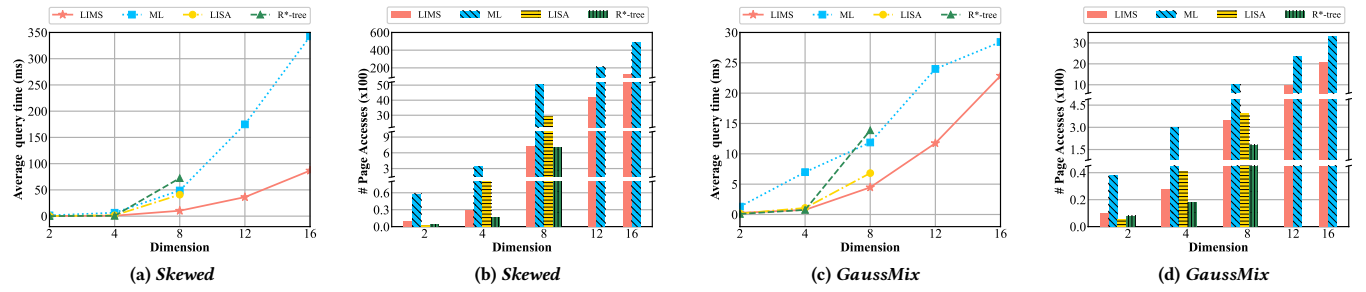


Figure 8: kNN query performance with dimensionality

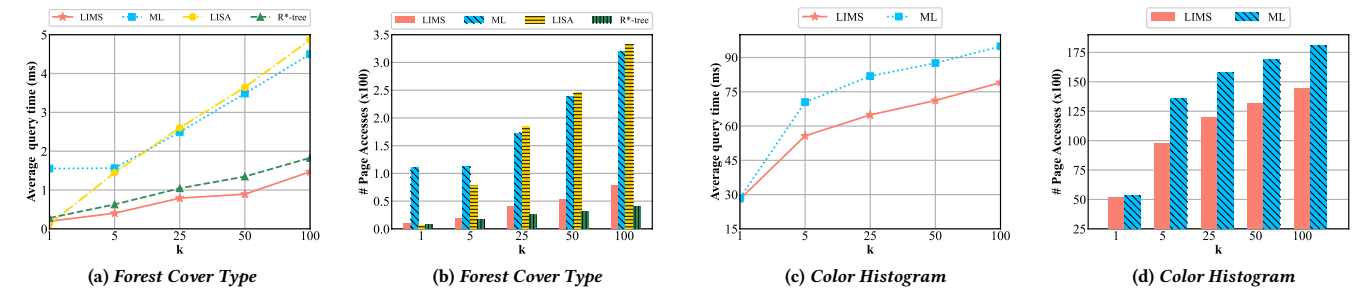


Figure 9: kNN query performance with k

6.4 k NN Query Performance

In this subsection, we study the performance of LIMS, ML, LISA, R^* -tree and M-tree on k NN query, as summarized in Figure 8, 9 and 11. ZM is excluded because it does not support k NN query.

Performance with dimensionality. Figure 8 (a)(b) and (c)(d) report the average query time and the number of page accesses on the *Skewed* and *GaussMix* datasets, respectively. k is set to 5. As expected, R^* -tree works best in low dimension, but it is less effective than LIMS with increased dimensionality. The relative performance of LIMS and ML is similar to range query, since both techniques follows similar search region expansion paradigm. Even though LISA uses a model learned from the data distribution to estimate initial radius, it still suffers from too many unnecessary page accesses. The reason is that, once the number of retrieved objects is smaller than k , LISA will issue a range query with larger radius from the scratch, leading to the same page accessed repeatedly. When the estimated initial radius is large, it also incurs many page accesses.

Performance with k . Figure 9 (a)(b) and (c)(d) show the performance when varying the value of k in $\{1, 5, 25, 50, 100\}$ on the *Forest Cover Type* and *Color Histogram* datasets, respectively. LIMS is the fastest except for 1NN on *Forest*, where LISA is slightly faster owing to a proper radius estimation. However, as the k grows, it becomes uncompetitive according to the aforementioned reasons. Consistent high efficiency of LIMS indicate that it scales to large k values.

Performance in generic metric spaces. Figure 11 present the performance on the *Signature* dataset. As expected, LIMS again yields fastest query time and fewest page accesses. The curve becomes gentle after $k = 5$ because many signatures share the same *edit distance* to a given signature.

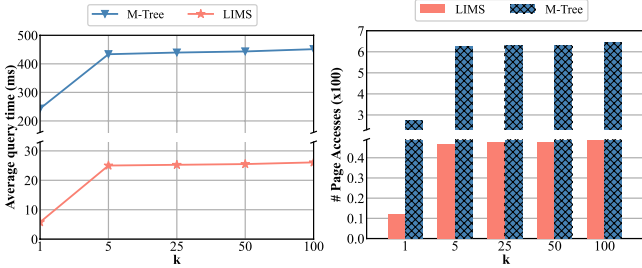


Figure 11: k NN query performance on *Signature*

6.5 Index Construction Time and Size

In this subsection, we report index construction time and size. As Figure 12 (a) shown, LIMS does not suffer from long training time, a common dilemma faced by learned indexes. On *8d GaussMix*, LIMS is 119X faster than LISA (47.6s v.s. 1.6h). Retraining the index for a cluster only takes an average of 0.476s, which makes LIMS easy to maintain and ensures simple update operations effective in practice. LIMS has a longer construction time than ZM and ML because of expensive distance computations in a generic metric space. However, LIMS has a decided advantage over ZM and ML, regardless of the dataset or dimension, when processing queries as discussed above. Figure 12 (b) shows index size, where LIMS has a slightly larger index size than other learned indexes because LIMS

needs to store the pre-computed distances between pivots and data objects. However, it is accepted because a few extra distance values are relatively insignificant compared to complex and large data, such as images and audio. And we expect LIMS construction time and index size to be smaller when using fewer pivots. The index size of LIMS is only 1/3 that of R^* -tree on *8d GaussMix* and 1/10 that of M-tree on *Signature*, respectively, since traditional indexes have to store a large number of routing (internal) nodes.

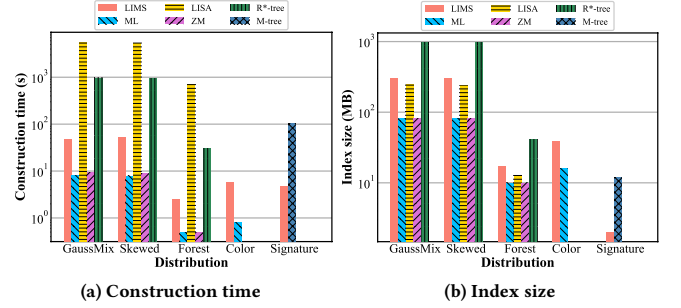


Figure 12: Index construction time and size with distribution

6.6 Updates

We finally examine the impact of data updates. Table 3 report the average query time and the number of page accesses for range query after inserting 5K, 10K, 20K and 40K objects into *8d 10-million GaussMix* and *Skewed* datasets. As expected, insertions cause query time increase since there are more points to query, and the index becomes less optimal. However, the rate of performance degradation is slow and steady. For example, on the *Skewed* dataset, after 40K insertions, LIMS still achieves 3.4X speedup compared to the next closest index (28.05ms v.s. 94.68ms). We also studied the impact of deletions but omit those due to negligible influence on query performance and space constraints.

Table 3: Range query performance after insertions

# inserted points		5K	10K	20K	40K
Query time (ms)	<i>GaussMix 8d</i>	10.13	10.78	11.03	11.22
	<i>Skewed 8d</i>	23.47	24.36	25.87	28.05
# page accesses	<i>GaussMix 8d</i>	934	946	971	1019
	<i>Skewed 8d</i>	2422	2488	2621	2884

7 CONCLUSIONS

As a universal abstraction of various data types, metric spaces and associated similarity search play an important role in many real-life applications. In this paper, we have developed LIMS, a novel learned index structure, for efficient exact similarity search query processing in metric spaces. LIMS takes advantage of compact-partitioning methods, pivot-based techniques and the idea of *learned index* to organize and access data. Experimental results suggest that LIMS responds significantly better to ‘curse of dimensionality’ compared with other learned index structures, and has a clear advantage over traditional indexes. In the future, we plan to take this work further by considering more relevant information such as query workload.

REFERENCES

- [1] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. In *SIGMOD*, 1990.
- [2] Benjamin Bustos, Oscar Pedreira, and Nieves Brisaboa. A dynamic pivot selection technique for similarity search. In *ICDEW*, 2008.
- [3] Kaushik Chakrabarti and Sharad Mehrotra. Local dimensionality reduction: A new approach to indexing high dimensional spaces. *PVLDB*, 89-100, 2000.
- [4] Edgar Chávez, Gonzalo Navarro, Ricardo Baeza-Yates, and José Luis Marroquín. Searching in Metric Spaces. *CSUR*, 33(3): 273-321, 2001.
- [5] Lu Chen, Yunjun Gao, Xinhua Li, Christian S Jensen, and Gang Chen. Efficient metric indexing for similarity search. In *ICDE*, 2015.
- [6] Lu Chen, Yunjun Gao, Xuan Song, Zheng Li, Xiaoye Miao, and Christian S Jensen. Indexing metric spaces for exact similarity search. *arXiv preprint arXiv:2005.03468* (2020).
- [7] Paolo Ciaccia, Marco Patella, and Pavel Zezula. M-Tree: An Efficient Access Method for Similarity Search in Metric Spaces. In *VLDB*, 1997.
- [8] Kenneth L Clarkson. Nearest-Neighbor Searching and Metric Space Dimensions. MIT Press, 2005.
- [9] Angjela Davitkova, Evica Milchevski, and Sebastian Michel. The ML-Index: A Multidimensional, Learned Index for Point, Range, and Nearest-Neighbor Queries. In *EDBT*, 2020.
- [10] Jialin Ding, Vikram Nathan, Mohammad Alizadeh, and Tim Kraska. Tsunami: A Learned Multi-Dimensional Index for Correlated Data and Skewed Workloads. In *VLDB*, 2020.
- [11] Navarro G. A guided tour to approximate string matching. *CSUR*, 33(1): 31-88, 2001.
- [12] Tu Gu, Kaiyu Feng, Gao Cong, Cheng Long, Zheng Wang, and Sheng Wang. The RLR-Tree: A Reinforcement Learning Based R-Tree for Spatial Data. *CoRR* abs/2103.04541 (2021). arXiv:2103.04541
- [13] Magnus Lie Hetland. The Basic Principles of Metric Indexing. Springer, 2009.
- [14] Dorit Hochbaum and David Shmoys. A Best Possible Heuristic for the k-Center Problem. *Mathematics of operations research*, 10(2): 180-184, 1985.
- [15] Hosagrahar V Jagadish, Beng Chin Ooi, Kian-Lee Tan, Cui Yu, and Rui Zhang. iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *TODS*, 30(2): 364-397, 2005.
- [16] Anil K Jain, M Narasimha Murty, and Patrick J Flynn. Data clustering: a review. *CSUR*, 31(3): 264-323, 1999.
- [17] Tim Kraska, Mohammad Alizadeh, Alex Beutel, H Chi, Ani Kristo, Guillaume Leclerc, Samuel Madden, Hongzi Mao, and Vikram Nathan. Sagedb: A learned database system. In *CIDR*, 2019.
- [18] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *SIGMOD*, 2018.
- [19] Pengfei Li, Hua Lu, Qian Zheng, Long Yang, and Gang Pan. LISA: A learned index structure for spatial data. In *SIGMOD*, 2020.
- [20] Rui Mao, Willard L Miranker, and Daniel P Miranker. Pivot selection: Dimension reduction for distance-based indexing. *Journal of Discrete Algorithms*, 13: 32-46, 2012.
- [21] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. Learning multi-dimensional indexes. In *SIGMOD*, 2020.
- [22] J. A. Orenstein and T. H. Merrett. A Class of Data Structures for Associative Searching. *Proceedings of the 3rd ACM SIGACT-SIGMOD Symposium on Principles of Database Systems*. 181-190, 1984.
- [23] B-U Pagel, Flip Korn, and Christos Faloutsos. Deflating the dimensionality curse using multiple fractal dimensions. *ICDE*, 589-598, 2000.
- [24] Oscar Pedreira and Nieves R Brisaboa. Spatial selection of sparse pivots for similarity search in metric spaces. *International Conference on Current Trends in Theory and Practice of Computer Science*, 434-445, 2007.
- [25] Martin L Puterman. Markov decision processes: discrete stochastic dynamic programming. John Wiley & Sons, 2014.
- [26] Jianzhong Qi, Guanli Liu, Christian S Jensen, and Lars Kulik. Effectively learning spatial indices. *PVLDB*, 13(12): 2341-2354, 2020.
- [27] Dmitri Rachkovskij. Distance-Based Index Structures for Fast Similarity Search. *Cybernetics and Systems Analysis*, 53(4): 636-658, 2017.
- [28] Y. Rubner, C. Tomasi, and L.J. Guibas. A metric for distributions with applications to image databases. *Sixth International Conference on Computer Vision*. 59-66, 1998.
- [29] Bernd Schröder. Ordered Sets: An Introduction with Connections from Combinatorics to Topology. Birkhäuser, 2016.
- [30] Yufei Tao, Man Lung Yiu, and Nikos Mamoulis. Reverse nearest neighbor search in metric spaces. In *TKDE*, 18(9): 1239-1252, 2006.
- [31] Robert L. Thorndike. Who belongs in the family? *Psychometrika*, 18: 267-276, 1953.
- [32] C. Traina, R. S. Filho, Agma J. M. Traina, Marcos R. Vieira, and C. Faloutsos. The Omni-family of all-purpose access methods: a simple and effective way to make similarity search more efficient. *The VLDB Journal*, 16(4): 483-505, 2005.
- [33] Jayendra Venkateswaran, Tamer Kahveci, Christopher Jermaine, and Deepak Lachwani. Reference-based indexing for metric spaces with costly distance measures. *The VLDB Journal*, 17(5): 1231-1251, 2008.
- [34] Haixin Wang, Xiaoyi Fu, Jianliang Xu, and Hua Lu. Learned index for spatial queries. In *MDM*, 2019.
- [35] Rui Xu and Donald Wunsch. Survey of clustering algorithms. *IEEE Transactions on neural networks*, 16(3): 645-678, 2005.
- [36] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. Qd-Tree: Learning Data Layouts for Big Data Analytics. In *SIGMOD*, 2020.
- [37] Pavel Zezula, Giuseppe Amato, Vlastislav Dohnal, and Michal Batko. Similarity Search - The Metric Space Approach. Springer Science & Business Media, 2006.